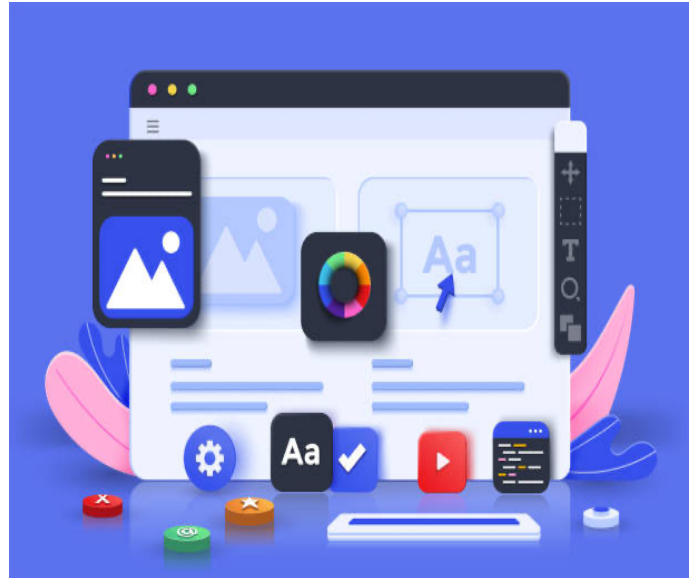


SD

—



Module Mijn eerste webgame

Instructie 07 – Ronde timer programmeren

- Minuten en Seconden met een voorloop nul?
- Verlopen minuten uit totaal aantal seconden berekenen

Auteur:	Johan Strootman
Afkorting:	SMN
Versie:	juli 2023

Inhoudsopgave

Inhoudsopgave	3
INLEIDING	4
Hoe gaan we deze functie nu z'n ding laten doen?	4
Het resultaat	5
Toelichting	5

INLEIDING

We willen nog de functie **roundsTimer** programmeren. We hebben in de **buttonClick** functie de **roundsTimer** functie gekoppeld aan een timer en laten de **roundsTimer** om de seconde uitvoeren. We hebben daarbij de functie wel gemaakt (om foutmeldingen te voorkomen) maar nog niet gevuld met code.

```
118     timer_id = setInterval( roundsTimer, 1000 );  
119 } else {
```

Figuur 1 - timer gestart in buttonClick functie

```
289  /*  
290  * In deze functie laten we steeds de ronde tijd op het scherm zien  
291  * Dit doen we in een eigen functie, omdat we de tijd nog moeten  
292  * vormgeven in de vorm 00:00. Dit is namelijk niet standaard.  
293  */  
294  function roundsTimer()  
295  {  
296  
297  }
```

Figuur 2 - Om foutmeldingen te voorkomen alvast een lege functie gemaakt

Hoe gaan we deze functie nu z'n ding laten doen?

Allereerst moeten we vaststellen wat de functie precies moet doen.

- Aantal verlopen seconden in de timer verhogen
Hulpvariabele **elapsedTimeInSeconds** gebruiken we daarvoor.
- Aantal verlopen minuten bepalen op basis van het totaal aantal verlopen seconden
Het totaal aantal verlopen seconden houden we bij in de hulpvariabele **elapsedTimeInSeconds**.
- De verlopen tijd in minuten en seconden tonen in de UI

Het resultaat

```
289  /*
290  * In deze functie laten we steeds de ronde tijd op het scherm zien
291  * Dit doen we in een eigen functie, omdat we de tijd nog moeten
292  * vormgeven in de vorm 00:00. Dit is namelijk niet standaard.
293  */
294  function roundsTimer()
295  {
296      let elapsedTimeInMinutes = 0; // Hulpvariabele
297
298      elapsedTimeInSeconds++; // Plus 1, want er is weer een seconde voorbij
299
300      // Het aantal verlopen minuten bepalen
301      elapsedTimeInMinutes = parseInt(elapsedTimeInSeconds / 60);
302
303      // We maken de weergave van de ronde tijd in de format 00:00
304      timer_info.innerHTML = (elapsedTimeInMinutes < 10 ? '0': '') + // 0 vooraf laten gaan in de minuten?
305                             elapsedTimeInMinutes.toString() + ':' +
306                             (parseInt((elapsedTimeInSeconds % 60)) < 10 ? '0': '') + // 0 vooraf laten gaan in de seconden?
307                             parseInt((elapsedTimeInSeconds % 60)).toString();
308  }
```

Figuur 3 - Eindresultaat

Toelichting

Regel 296

We maken hier weer een hulpvariabele aan, maar doordat we deze in de functie aanmaken blijft deze hulpvariabele alleen bestaan zolang de functie z'n werk doet. In deze variabele bewaren we het aantal verlopen minuten van de timer.

Regel 298

Iedere seconde wordt deze functie uitgevoerd. Dus kunnen we het aantal seconden in de globale hulpvariabele steeds met 1 verhogen.

Regel 301

Op deze regel berekenen we uit het totaal aantal seconden het aantal verlopen minuten. Dit doen we door het totaal aantal seconden te delen door 60 (het aantal seconden in een minuut). Maar probleem is dat dit mogelijkwerwijs een komma getal oplevert. En dat willen we natuurlijk niet, dus gebruiken we de JavaScript functie **parseInt**. Met deze functie krijgen we altijd een heel getal uit een komma getal.

Voorbeeld:

10.5678 levert met **parseInt** gewoon 10 op.

Regel 304 t/m 307

Dit lijkt complex, maar neem het maar eens aandachtig en rustig door dan zul je begrijpen wat hier gebeurt.

Laten we eens beetje bij beetje analyseren wat hier precies gebeurt.

```
(elapsedTimeInMinutes < 10 ? '0': '') +
```

Figuur 4 - Bepalen of er een 0 voor de minuten geplaatst moet worden

Met de ternary operator hierboven controleren we of het aantal minuten kleiner is dan 10. Als dat zo is willen we namelijk eerst een 0 vooraf aan het aantal minuten laten gaan. Dit levert dan een string op, zoals b.v. "05" of "23". Met de + aan het eind vertellen we dat we er nog een andere string aan vast willen plakken.

```
elapsedTimeInMinutes.toString() + ':' +
```

Figuur 5 - Dubbele punt erachter plakken

In de hulpvariabele `elapsedTimeInMinutes` zit een getal en dus geen string. Met de method `.toString()` kunnen we van een getal een string maken.

Met + ':' plakken we aan het aantal minuten een dubbele punt vast. Met de laatste + vertellen we dat ook hier nog een andere string aan vast willen plakken.

```
(parseInt((elapsedTimeInSeconds % 60)) < 10 ? '0': '') +
```

Figuur 6 - Bepalen of er een 0 voor de seconden moet worden geplaatst

Hierboven zien we code waarmee ook de seconden door een 0 laten voorafgaan wanneer het aantal seconden kleiner is dan 10. Ook nu weer maken we gebruik van een Ternary Operator.

```
parseInt((elapsedTimeInSeconds % 60)).toString();
```

Figuur 7 - Seconden tot slot eraan vastplakken

Tot slot plakken we nog het werkelijk aantal verlopen seconden aan het geheel vast.